

Linux :)

폴더구조

남궁명수

[1] 폴더구조.....	3
[1-1] / (Root Directory).....	3
[1-2] /bin (Binary / Executable)	3
[1-3] /sbin (System Binary).....	4
[1-4] /lib (Library)	4
[1-5] /boot	5
[1-6] /dev(Device).....	6
[1-7] /etc(Etcetera / Editable Text Config).....	7
[1-8] /home.....	8
[1-9] /media	9
[1-10] /mnt	10
[1-11] /proc(Process)	11
[1-12] /sys.....	12
[1-13] /run	14
[1-14] /srv(Service)	15
[1-15] /var(Variable).....	16
[1-16] /tmp(Temporary)	18
[1-17] /opt	20
[1-18] 리눅스 파일 시스템 구조	21

[1] 폴더구조

[1-1] / (Root Directory)

- 전체 파일 시스템의 시작 지점
- 리눅스의 모든 디렉터리와 파일은 / 아래에 존재
- 파일 시스템의 최상위 디렉터리(root directory) 역할 수행
- 다른 모든 디렉터리의 기준이 되는 디렉터리

예시:

```
/
├── bin
├── etc
├── home
└── usr
```

[1-2] /bin (Binary / Executable)

- 시스템이 정상적으로 동작하는 데 필요한 명령어들이 저장되는 디렉터리
- 부팅 과정이나 복구 모드에서도 반드시 사용되는 핵심 명령 포함
- 파일 시스템의 다른 영역이 아직 마운트되지 않았더라도 사용할 수 있도록 구성됨

포함되는 대표 명령어

- ls : 파일 및 디렉터리 목록 확인
- cp: 파일 복사
- mv: 파일 이동 및 이름 변경
- cat: 파일 내용 출력

특징

- 기본적인 파일 관리와 텍스트 확인 작업 수행 가능
- 시스템 관리 및 문제 해결 과정에서 매우 중요하게 사용됨
- 최소한의 환경에서도 동작해야 하는 필수 실행 파일 저장

정리

- /bin은 리눅스의 핵심 실행 명령어 저장소
- 시스템 운영과 복구를 위해 항상 접근 가능해야 하는 디렉터리
- 일반적으로 사용자가 직접 수정하지 않으며, 잘못 수정할 경우 시스템 핵심 기능에 문제가 발생할 수 있음

[1-3] /sbin (System Binary)

- 시스템 관리자(root)를 위한 필수 시스템 관리 명령어가 저장되는 디렉터리
- 시스템 유지보수 및 복구 작업에 사용되는 핵심 유틸리티 포함
- 일반 사용자보다 관리자 권한(root)이 필요한 명령어가 대부분 존재

포함되는 대표 명령어

- mount: 파일 시스템 마운트
- fsck: 파일 시스템 검사 및 복구
- shutdown: 시스템 종료

특징

- 시스템 부팅, 복구, 유지보수 과정에서 매우 중요하게 사용됨
- 디스크 관리 및 시스템 상태 점검과 같은 하위 수준 작업 수행
- 대부분 관리자 권한(root)이 필요함

정리

- /sbin은 시스템 관리용 핵심 실행 파일 저장소
- 일반 사용자가 직접 사용할 일은 많지 않음
- 잘못 수정하거나 사용하면 시스템에 심각한 문제가 발생할 수 있으므로 주의해야 함

[1-4] /lib (Library)

- /bin, /sbin에 있는 명령어들이 실행될 때 필요한 공유 라이브러리 저장
- 여러 프로그램이 공통으로 사용하는 기능을 제공하는 파일들이 위치함
- 파일 입출력(I/O), 메모리 처리 등의 공통 기능 제공

특징

- 동일한 코드를 각 프로그램마다 포함하지 않아도 되므로 효율적
- 프로그램 실행 시 필요한 핵심 라이브러리 저장
- 시스템에 따라 다음과 같은 디렉터리가 존재할 수 있음
 - /lib32: 32비트 라이브러리
 - /lib64: 64비트 라이브러리

커널 모듈

- /lib에는 커널 모듈(kernel module)도 저장됨
- 커널 모듈은 필요할 때 커널에 동적으로 로드되는 코드 조각
- 하드웨어 드라이버나 파일 시스템 기능 등을 확장하는 역할 수행

정리

- /lib는 시스템 명령어 실행에 필요한 핵심 라이브러리 저장소
- 커널 기능 확장을 위한 모듈도 포함됨
- 일반 사용자가 수정해서는 안 되며, 잘못 수정하면 시스템 부팅이나 기본 명령 실행에 문제가 발생할 수 있음

[1-5] /boot

- 시스템 부팅에 필요한 핵심 파일들이 저장되는 디렉터리
- 리눅스 커널과 부트로더 관련 파일 포함
- 운영체제가 시작되기 전에 가장 먼저 사용되는 영역 중 하나

포함되는 주요 파일

- Linux Kernel
- initrd / initramfs
- GRUB 부트로더 파일
- grub.cfg

주요 역할

(1) Linux Kernel

- 운영체제의 핵심 프로그램
- 하드웨어와 소프트웨어를 연결하는 역할 수행

(2) initrd / initramfs

- 실제 루트 파일 시스템이 마운트되기 전에 필요한 임시 파일 시스템
- 부팅 초기 단계에서 드라이버 및 필수 기능 로드

(3) GRUB

- 대표적인 리눅스 부트로더
- 어떤 운영체제를 어떤 방식으로 부팅할지 결정

(4) grub.cfg

- GRUB의 설정 파일
- 커널 위치, 부팅 옵션 등을 정의
- 시스템이 운영체제를 어떻게 로드할지 지정

특징

- 일부 시스템은 /boot를 별도 파티션으로 분리하여 사용
- 특히 다음과 같은 환경에서 자주 사용됨
 - 듀얼 부팅
 - 디스크 암호화 사용
 - 복구 및 부팅 안정성 확보

정리

- /boot는 시스템 부팅에 필요한 핵심 파일 저장소
- 매우 중요한 디렉터리이므로 일반 사용자가 직접 수정하지 않는 것이 좋음
- 잘못 수정하면 시스템이 부팅되지 않음

[1-6] /dev(Device)

- 하드웨어 장치를 파일 형태로 관리하는 디렉터리
- 일반 파일이 아닌 장치 파일(device file) 저장
- 리눅스의 “모든 것은 파일이다”라는 개념이 적용되는 대표적인 영역

특징

- 시스템의 하드웨어 정보를 파일 형태로 제공
 - 하드디스크, USB, 키보드 같은 실제 장치뿐 아니라 가상 장치도 포함
- 예시: /dev/sda - 첫 번째 저장장치(하드디스크, SSD 등)를 의미할 수 있음

포함되는 장치 예시

장치 파일	설명
/dev/sda	첫 번째 저장장치
/dev/tty	터미널 장치
/dev/null	데이터를 버리는 가상 장치
/dev/random	난수 생성 장치

장치 파일의 종류

(1) Block Device(블록 장치)

- 데이터를 블록(덩어리) 단위로 처리
- 주로 저장장치에 사용
- HDD, SSD, USB 저장장치

(2) Character Device(문자 장치)

- 데이터를 문자 스트림 형태로 처리
- 연속적인 입력/출력 장치에서 사용
- 키보드, 마우스, 터미널 장치

동작 방식

- 장치 파일은 동적으로 생성 및 제거됨
- 하드웨어 연결 상태에 따라 리눅스가 자동 관리
 - USB 연결: /dev/sdb 생성
 - USB 제거: 해당 장치 파일 제거

정리

- /dev는 하드웨어 장치를 파일 형태로 제공하는 디렉터리
- 리눅스에서 장치와 상호작용하는 핵심 영역
- 직접 수정하거나 잘못 접근하면 하드웨어 및 시스템에 영향을 줄 수 있으므로 주의해야 함

[1-7] /etc(Etcetera / Editable Text Config)

- 시스템 전체의 설정 파일(configuration file)을 저장하는 디렉터리
- 네트워크 설정, 사용자 계정, 서비스 설정 등 시스템 운영에 필요한 구성 정보 포함
- 대부분 일반 텍스트 파일 형태로 존재

주요 역할

- 시스템 전반의 환경 및 동작 방식 설정
- 부팅 과정과 서비스 실행 설정 관리
- 사용자 및 네트워크 관련 설정 저장

대표 파일

파일	설명
/etc/passwd	사용자 계정 정보 저장
/etc/fstab	부팅 시 파일 시스템 마운트 설정
/etc/hosts	호스트 이름과 IP 주소 매핑
/etc/resolv.conf	DNS 설정

특징

- 시스템 관리자가 읽고 수정할 수 있도록 설계됨
- 대부분 사람이 읽을 수 있는 텍스트 파일 형태
- 시스템 동작에 직접적인 영향을 주는 중요한 설정 저장

etc의 의미

- 원래 “기타 등등(Et cetera)”에서 유래
- 현재는 “Editable Text Config”로 이해하기로 함

정리

- /etc는 시스템 설정 파일의 중앙 저장소
- 리눅스의 동작 방식을 이해하는 데 매우 중요한 디렉터리
- 잘못 수정하면 시스템 전체 동작에 영향을 줄 수 있으므로 신중하게 다루어야 함

[1-8] /home

- 일반 사용자의 개인 파일과 설정을 저장하는 디렉터리
- 각 사용자는 자신의 홈 디렉터리(home directory)를 가짐

예시:

/home/alex

/home/emma

- 문서, 다운로드, 프로젝트, 개인 설정 파일 등을 저장

특징

- 사용자 데이터를 시스템 파일과 분리하여 관리
- 백업 및 사용자 계정 관리가 쉬워짐
- 사용자는 자신의 홈 디렉터리 안에서 자유롭게 파일 생성 및 수정 가능

사용자 설정 파일

- 사용자별 환경 설정 파일도 홈 디렉터리에 저장됨
- 대부분 숨김 파일(hidden file) 형태로 존재

예시:

.bashrc

.profile

- 리눅스에서 .으로 시작하는 파일은 숨김 파일을 의미

권한 및 보안

- 기본적으로 다른 사용자의 /home 디렉터리에 접근 불가능
- 권한(permission)을 변경해야만 접근 가능
- 사용자 간 개인정보 보호 기능 제공

공유 폴더

- 일부 시스템에서는 사용자 간 파일 공유를 위한 공용 디렉터리가 존재할 수 있음

예시:

/home/shared

root 사용자의 홈 디렉터리

- 일반 사용자와 달리 root 사용자는 별도의 홈 디렉터리 사용

예시

/root

- /home/root를 사용하지 않음
- 관리자 계정을 일반 사용자 영역과 분리하여 보안성과 격리 강화

정리

- /home은 일반 사용자의 작업 공간
- 개인 파일과 사용자 설정 저장
- 사용자별 독립된 환경 제공
- 시스템 디렉터리와 달리 자유롭게 사용 및 수정 가능한 영역임

[1-9] /media

- USB, CD, DVD 같은 이동식 저장장치를 자동으로 마운트하는 디렉터리
- 사용자가 실제 파일과 데이터에 접근하는 위치
- 최신 리눅스 데스크톱 환경에서 자동으로 관리하는 경우가 많음

특징

- 이동식 장치마다 별도의 하위 디렉터리 생성
- 디렉터리 이름은 장치의 볼륨 이름(label)을 사용하는 경우가 많음

예시:

/media/USB

/media/MyDrive

- 사진, 문서, 영상 등 실제 저장된 파일 접근 가능
- 일반적으로 관리자 권한 없이 사용 가능

데스크톱 환경과의 연동

- GNOME, KDE 등의 데스크톱 환경이 자동 처리
- USB 연결 시 자동 마운트되어 바로 사용 가능

예시

- USB 연결 -> /media/USB 생성 -> 파일 탐색기에서 바로 접근 가능

/media vs /dev

(1) /dev: 장치 자체를 나타냄(하드웨어 연결 지점)

- 하드웨어 장치 자체를 나타내는 장치 파일 저장
- 시스템과 하드웨어를 연결하는 원시 장치 정보 제공

예시:

/dev/sdb1

(2) /media: 장치 내용을 사용할 수 있게 연결된 위치

- 장치 안의 실제 파일에 접근할 수 있도록 마운트된 위치

예시:

/media/USB

정리

- /media는 이동식 저장장치를 자동 마운트하는 디렉터리
- 사용자가 USB나 CD 내부 파일에 쉽게 접근할 수 있도록 제공
- 현대 리눅스 시스템에서는 대부분 자동으로 관리됨

[1-10] /mnt

- 파일 시스템을 수동으로 마운트할 때 사용하는 디렉터리
- 주로 시스템 관리자(root)가 임시 저장장치나 파일 시스템을 연결할 때 사용
- 유지보수, 복구, 테스트 작업에서 자주 사용됨

특징

- 기본적으로 비어 있는 경우가 많음
- 필요할 때만 특정 장치를 임시로 연결하여 사용
- 자동 마운트보다 수동 마운트 목적에 적합

예시:

/mnt/backup

/mnt/recovery

사용 예시

- USB 저장장치 수동 마운트
- 손상된 디스크 복구 작업
- 네트워크 공유 연결
- 백업 디스크 접근
- 시스템 점검 및 유지보수

예시 명령:

```
mount /dev/sdb1 /mnt
```

/media vs /mnt

- /media: 이동식 장치를 자동 마운트
- /mnt: 관리자가 수동으로 임시 마운트

차이점 설명

(1) /media

- 일반 사용자를 위한 자동 마운트 공간
- USB 연결 시 자동으로 접근 가능
- 데스크톱 환경(GNOME, KDE 등)이 자동 처리

(2) /mnt

- 관리자가 직접 마운트하는 공간
- 유지보수 및 복구 작업에 자주 사용
- 스크립트나 서버 환경에서도 활용

정리

- /mnt는 임시 파일 시스템 마운트를 위한 디렉터리
- 주로 관리자나 고급 사용자 작업에서 사용됨
- 자동 마운트용 /media와 목적이 다름
- 시스템 관리 및 복구 과정에서 매우 유용한 디렉터리임

[1-11] /proc(Process)

- 리눅스 커널과 실행 중인 프로세스 정보를 제공하는 가상 파일 시스템(Virtual File System)
- 실제 디스크에 저장된 파일이 아니라, 커널이 메모리상에서 동적으로 생성하는 정보
- 시스템 상태를 실시간으로 확인할 수 있는 인터페이스 역할 수행

특징

- 시스템 및 프로세스 정보를 실시간으로 제공
- 파일처럼 보이지만 실제 저장 파일 아님
- 커널 내부 상태를 사용자 공간에서 확인 가능

/proc vs /dev

- /dev : 하드웨어 장치 인터페이스, 장치 접근용
- /proc : 커널 및 프로세스 정보 인터페이스, 시스템 상태 확인용

프로세스 정보

- 실행 중인 각 프로세스마다 PID(Process ID) 이름의 디렉터리 생성

예시:

/proc/1234

- 해당 프로세스의 상태, 메모리 사용량, 실행 정보 등을 확인 가능

주요 파일 시스템 정보 파일

파일	설명
/proc/cpuinfo	CPU 정보
/proc/meminfo	메모리 정보
/proc/uptime	시스템 가동 시간
/proc/version	커널 버전 정보

동작 방식

- 커널이 필요한 순간에 정보를 생성
- 시스템 상태 변화에 따라 내용이 실시간으로 변경됨

정리

- /proc은 커널과 프로세스 정보를 제공하는 가상 파일 시스템
- 시스템 상태 모니터링과 디버깅에 매우 중요함
- 리눅스 내부 동작을 이해하는 핵심 디렉터리 중 하나

[1-12] /sys

- 커널과 하드웨어 사이의 정보를 제공하는 가상 파일 시스템
- sysfs(System File System)의 일부
- 실제 디스크에 저장된 파일이 아니라 커널이 동적으로 생성하는 정보 제공

특징

- 하드웨어 장치와 커널 정보를 계층 구조 형태로 제공
- 장치, 드라이버, 시스템 버스 등의 상태 확인 가능
- 일부 파일은 읽기뿐 아니라 쓰기(write)도 가능

즉,

- 단순 정보 조회뿐 아니라 커널 설정을 실시간으로 변경할 수 있다.

포함되는 정보 예시

- CPU 정보
- 메모리 블록 정보
- USB 장치 정보
- 전원 상태 정보
- 드라이버 정보

예시:

/sys/class

/sys/block

/sys/devices

/proc vs /sys

- /proc: 프로세스 및 커널 상태 정보, 시스템 동작 상태 중심
- /sys: 하드웨어 및 장치 정보, 하드웨어 및 드라이버 중심

쓰기 가능한 파일

- /sys의 일부 파일은 값을 변경 가능
- 커널 동작이나 하드웨어 설정을 실시간 조정 가능

예시:

- CPU 절전 모드 변경
- 장치 상태 제어
- 전원 관리 설정 변경

시스템 유틸리티와의 관계

다음과 같은 시스템 도구들이 /sys를 사용함

도구	역할
udev	장치 관리
전원 관리자	배터리 및 전력 관리
하드웨어 설정 도구	장치 상태 제어

구조 특징

- 계층 구조(tree structure) 형태로 구성
- 하드웨어 장치마다 고유 경로 존재

예시:

/sys/devices/

/sys/class/net/

/sys/bus/usb

정리

- /sys는 커널과 하드웨어 사이의 인터페이스 역할 수행
- 하드웨어 상태 조회 및 설정 변경 가능
- 시스템 유틸리티가 내부적으로 많이 사용하는 핵심 디렉터리
- 일반 사용자가 직접 수정하는 경우는 드물며, 잘못 변경하면 시스템 동작에 영향을 줄 수 있음

[1-13] /run

- 시스템과 프로그램이 실행 중(runtime)에 사용하는 임시 데이터를 저장하는 디렉터리
- 메모리 기반(tmpfs)으로 동작하는 경우가 많음
- 시스템 부팅 초기에 생성되며 재부팅 시 내용이 모두 사라짐

특징

- 실행 중인 프로세스들의 상태 정보 저장
 - 프로세스 간 통신(IPC)에 사용
 - PID, 소켓(socket), 잠금(lock) 파일 등 짧은 수명의 데이터 저장
- 즉, 시스템이 현재 실행 중일 때만 필요한 임시 데이터 저장 공간

주요 하위 디렉터리

디렉터리	설명
/run/user	사용자 세션 관련 정보
/run/lock	시스템 잠금(lock) 파일
/run/systemd	systemd 런타임 정보

예시

/run/user
/run/lock

/proc, /sys와의 차이

- /proc: 프로세스 및 커널 정보 제공 -> 정보 조회 중심
- /sys: 하드웨어 및 장치 정보 제공 -> 정보 조회 중심
- /run: 실행 중인 프로그램의 임시 런타임 데이터 저장 -> 실제 프로그램이 데이터를 기록하며 사용

저장되는 데이터 예시

- PID(Process ID) 파일
- 소켓(socket) 파일
- 잠금(lock) 파일
- 세션 정보

/run/nginx.pid

- 실행 중인 nginx 프로세스의 PID 저장

기존 디렉터리 대체

/run은 다음 디렉터리들을 통합하여 대체함

- /var/run -> /run
- /var/lock -> /run/lock

이를 통해 런타임 데이터를 하나의 중앙 위치에서 관리 가능

정리

- /run은 시스템 실행 중 필요한 임시 런타임 데이터를 저장하는 디렉터리
- 재부팅하면 내부 데이터는 모두 사라짐
- 프로세스 상태 및 관리 및 프로세스 간 통신에 매우 중요함
- 현대 리눅스 시스템에서 핵심적인 런타임 관리 영역이다.

[1-14] /srv(Service)

- 시스템이 제공하는 서비스(Service)의 데이터가 저장되는 디렉터리
- 웹 서버, FTP 서버 등의 서비스가 사용하는 실제 콘텐츠 저장
- 네트워크를 통해 사용자에게 제공되는 데이터가 위치하는 공간

특징

- 서비스 운영에 필요한 데이터 저장
- 설정 파일이 아니라 실제 서비스 콘텐츠 저장
- 서버 환경에서 자주 사용됨

예시:

/srv/www

/srv/ftp

사용 예시

(1) 웹 서버

/srv/www

- 웹사이트 HTML 파일
- 이미지
- CSS/JS 파일 등 저장 가능

(2) FTP

/srv/ftp

- FTP를 통해 공유되는 파일 저장 가능

/etc vs /srv

- /etc : 서비스 설정 파일 저장 -> 서비스 동작 방식 설정
- /srv: 서비스가 제공하는 실제 데이터 저장 -> 사용자가 실제로 접근하는 콘텐츠 저장

주의할 점

- 모든 리눅스 배포판이 /srv를 적극적으로 사용하는 것은 아니다
- 일부 시스템은 서비스 데이터를 /var 아래에 저장하기도 한다.

정리

- /srv는 서비스가 제공하는 실제 데이터를 저장하는 디렉터리
- 서버 환경에서 웹사이트, FTP 데이터 등을 저장하는데 사용됨
- 서비스 관련 콘텐츠를 체계적으로 관리하기 위한 공간
- 실행 중인 서비스가 없다면 비어 있을 수도 있음

[1-15] /var(Variable)

- 시스템이 실행 중 자주 변경되는 데이터를 저장하는 디렉터리
- 로그(log), 캐시(cache), 스푼(spool), 데이터베이스 등의 가변 데이터 저장
- 시스템이 동작하면서 지속적으로 내용이 변경됨

특징

- 시스템 활동 기록 저장
- 프로그램 실행 중 생성되는 데이터 관리
- 시간이 지나면서 파일 크기가 커질 수 있음

주요 하위 디렉터리

디렉터리	설명
/var/log	시스템 로그 저장
/var/cache	캐시 데이터 저장
/var/spool	대기 작업(mail, print 등) 저장
/var/lib	프로그램 상태 및 메타데이터 저장
/var/tmp	장기간 유지되는 임시 파일 저장

/var/log

- 시스템 로그 저장
- 오류 및 시스템 활동 기록

예시:

/var/log/syslog

/var/log/auth.log

- 로그 파일은 용량 관리를 위해 주기적으로 순환(log rotation)됨

/var/lib

- 프로그램 상태 정보 및 메타데이터 저장
- 패키지 관리자 등이 사용

예시:

/var/lib/dpkg

- Debian 계열 패키지 정보 저장

특징 및 관리

- /var 데이터는 지속적으로 증가 가능
- 서버 환경에서는 용량 부족 방지를 위해 별도 파티션으로 분리하는 경우가 많음

예시

- 로그 파일 증가
- 데이터베이스 캐시 증가
- 메일 스푼 증가

보안

- 시스템 활동 정보와 사용자 관련 데이터가 저장될 수 있음
- 다중 사용자 및 서버 환경에서는 적절한 권한 관리 필요

정리

- /var는 시스템에서 자주 변경되는 데이터를 저장하는 디렉터리
- 로그, 캐시, 프로그램 상태 데이터 등을 포함
- 시스템 운영 및 문제 해결에서 매우 중요한 역할 수행
- 디스크 공간 관리와 보안 측면에서도 중요한 디렉터리이다.

[1-16] /tmp(Temporary)

- 임시 파일(temporary file)을 저장하는 디렉터리
- 프로그램이나 사용자가 작업 중 잠시 필요한 데이터를 저장하는 공간
- 일반적으로 재부팅 시 또는 일정 시간이 지나면 자동 삭제됨

특징

- 단기 사용 목적
- 모든 사용자가 쓰기 가능
- 임시 작업 파일 저장에 사용

예시:

- 프로그램 설치 중 생성되는 임시 파일
- 문서 편집 중 생성되는 임시 저장 파일
- 압축 해제 중 생성되는 작업 파일

사용 예시

/tmp/install.tmp
/tmp/editor_backup

Sticky Bit(스티키 비트)

- /tmp는 모든 사용자가 쓰기 가능하지만, 특별한 권한인 Sticky Bit가 적용됨

예시:

drwxrwxrwt
- t: Sticky Bit 의미

Sticky Bit 역할

- 자신이 생성한 파일만 삭제 가능
 - 다른 사용자의 파일 삭제 방지
 - 공유 디렉터리 환경에서 안전성 제공
- 즉, 모두가 파일 생성 가능, 하지만 파일 소유자만 삭제 가능

tmpfs와 메모리 사용

- 일부 시스템에서는 /tmp와 tmpfs로 마운트됨
- 디스크 대신 RAM 사용
 - 매우 빠른 속도 제공
 - 하지만, 큰 파일 저장 시 메모리(RAM) 사용량 증가 가능

특징 요약

항목	설명
용도	임시 파일 저장
저장 기간	짧음
재부팅 시	대부분 삭제
권한	모든 사용자 쓰기 가능
보호 방식	Sticky Bit 적용
일부 시스템	tmpfs(RAM 기반) 사용

정리

- /tmp는 임시 데이터를 저장하는 공간
- 프로그램 실행 과정에서 자주 사용됨
- 일반적으로 자동 정리됨
- Sticky Bit를 통해 사용자 파일 보호
- 메모리 기반(tmpfs) 환경에서는 과도한 사용에 주의해야 함

[1-17] /opt

- 선택적(optional) 소프트웨어나 타사(third-party) 프로그램을 설치하는 디렉터리
- 시스템 기본 프로그램과 분리하여 관리되는 공간
- 주로 대형 응용 프로그램이나 독점 소프트웨어 설치에 사용됨

특징

- 각 프로그램이 자신의 하위 디렉터리를 가짐
- 프로그램별로 독립적으로 관리 가능
- 제거 및 유지보수가 쉬움

예시:

/opt/google/chrome

/opt/discord

사용 목적

- 일반 패키지 관리자 경로와 분리된 소프트웨어 저장
- 기업용 프로그램이나 상용 프로그램 설치 시 자주 사용
- 시스템 핵심 파일과 충돌을 줄이기 위한 공간 제공

특징 설명

일반적인 리눅스 프로그램은 보통 다음 위치를 사용함

/usr/bin

/usr/lib

하지만 /opt 프로그램들은

- 자체 구조를 유지하는 경우가 많음
 - 표준 리눅스 파일 구조를 완전히 따르지 않을 수도 있음
- 즉, 프로그램 전체를 하나의 독립된 폴더처럼 관리한다.

장점

- 독립성: 시스템 파일과 분리
- 관리 편의성: 설치/삭제 쉬움
- 충돌 감소: 기존 시스템과 분리 운영

권한

- 대부분 관리자(root) 권한 필요
- 시스템 영역이므로 일반 사용자가 수정하는 경우는 드뭄

정리

- /opt는 선택적 소프트웨어 및 타사 프로그램 설치 공간
- 프로그램별 독립 구조를 유지하기 쉬움
- 기업용 및 상용 소프트웨어에서 자주 사용됨
- 시스템 핵심 파일과 분리되어 안전하게 관리 가능

[1-18] 리눅스 파일 시스템 구조

리눅스 파일 시스템 구조는 단순한 폴더들의 집합이 아니다.

시스템 구성 요소, 사용자 데이터, 임시 파일, 프로그램 리소스를 체계적으로 분리하기 위해 설계된 계층 구조이다.

각 디렉터리는 고유한 목적을 가지며, 이를 통해 시스템의 안정성, 보안, 유지보수성을 높인다.

예를 들어,

- /bin, /sbin -> 시스템 핵심 명령어
- /etc -> 설정 파일
- /home -> 사용자 데이터
- var -> 로그 및 가변 데이터
- tmp -> 임시 파일
- /proc, /sys -> 커널 및 시스템 정보

이처럼 역할이 명확하게 분리되어 있기 때문에 시스템 관리가 쉬워지고 문제 발생 시 원인 파악이 빨라지며, 백업 및 보안 관리도 효율적으로 수행할 수 있다.

또한, 파일 구조를 이해하면

- 리눅스 명령어 활용 능력 향상
- 시스템 관리 및 서버 운영
- 셸 스크립트 작성
- 디버깅 및 문제 해결
- 보안 및 권한 관리

와 같은 고급 작업의 기초를 탄탄하게 다질 수 있다.

마운트(mount)

USB, 하드디스크, CD/DVD 등 물리적 저장 장치를 운영체제(리눅스, 유닉스)의 파일 시스템 디렉터리 구조(폴더)에 연결하여, 사용자가 해당 장치 내의 파일에 접근할 수 있게 준비된 상태를 뜻한다.

부트로더(Boot Loader)

기기가 켜질 때 가장 먼저 실행되는 프로그램으로, 하드웨어를 초기화하고 운영체제(OS)나 메인 펌웨어를 메모리에 적재(Load)하여 실행하는 역할을 담당하는 컴퓨터 프로그램이다.

GNOME(그놈)

리눅스 및 유닉스 계열 운영체제에서 가장 널리 사용되는 개방형 데스크톱 환경(GUI)이다.

KDE

마우스와 아이콘으로 컴퓨터를 제어하는 그래픽 사용자 인터페이스(GUI)이다.

FTP(파일 전송 프로토콜)

컴퓨터 네트워크를 통해 서버와 클라이언트 간에 파일을 전송하는데 사용되는 표준 통신 규약이다.

Spool(스풀)

중앙처리장치(CPU)와 속도가 느린 주변장치(주로 프린터) 간의 속도 차이를 해결하기 위해 사용하는 데이터 임시 저장 기법이다.